

SYSTEM ARCHITECTURE DOCUMENT

CivicVox-Omni

Edge-compute emergency intelligence — full system design reference

Version: 1.0 — DRAFT **Date:** May 29, 2026

1. System Overview

CivicVox-Omni is a four-layer, local-first edge application. Every computation — speech transcription, vision inference, geospatial query, and report generation — runs on a single ASUS GX10 node (NVIDIA GB10 Grace Blackwell Superchip) with zero cloud API dependency during live operation. The four layers are:

#	Layer	Responsibility	Primary Tech
1	Input Stream	Capture live video/audio/GPS from browser or device	HTML5 MediaDevices API, WebRTC / WebSocket
2	API Gateway	Validate payloads, run STT, route to agent engine, return results	FastAPI, Uvicorn, Faster-Whisper, Kokoro TTS
3	Agent Orchestration	Multi-agent state machine: orchestrate, analyse, locate, compile	LangGraph, LangChain, GeoPandas, ChromaDB
4	AI Inference + Data	Serve VLM on GB10; spatial queries over Toronto Open Data	NVIDIA NIM / vLLM, TensorRT-LLM, GeoPandas R-tree

2. Layer 1 — Input Stream

2.1 Browser capture

The frontend (Next.js, running on the user's laptop or phone browser) handles all media capture using standard Web APIs:

- Video: `navigator.mediaDevices.getUserMedia({ video: true })` → `canvas.drawImage()` → JPEG base64 snapshot every 2 000 ms
- Audio: MediaRecorder API → PCM blob → sent alongside each video frame
- GPS: `navigator.geolocation.getCurrentPosition()` → { lat, lng } JSON payload

2.2 Transport to gateway

Two transport modes are supported — the frontend switches automatically based on session type:

Mode	Endpoint	When used
HTTP polling	POST /api/incident (every 2 s)	Default; simple, reliable on local Wi-Fi
WebSocket stream	WS /ws/stream (continuous)	Demo mode: enables live agent telemetry events to dashboard

3. Layer 2 — API Gateway

The gateway (backend/server.py) is a FastAPI application bound to 0.0.0.0:8080. It is the only network-accessible process; all other components communicate locally through Python function calls.

3.1 Endpoint specifications

Method	Endpoint	Input schema	Output schema
POST	/api/incident	{ transcript, frame_b64, gps: {lat, lng} }	{ report, vision, spatial, urgency }
WebSocket	/ws/stream	Binary audio + JSON metadata frames	Server-sent JSON: { node, status, timestamp }
POST	/api/synthesize	{ text: string }	WAV audio bytes
GET	/api/health	—	{ status, models_loaded }

3.2 Speech pipeline (Faster-Whisper)

- Model: large-v3, loaded in float16 on CUDA at server startup
- Function: transcribe_audio(pcm_bytes: bytes) -> str
- Target latency: < 300 ms for a 10-second audio clip on GB10
- Output transcript is injected into the AgentState.messages field before invoking the LangGraph engine

3.3 TTS pipeline (Kokoro-82M)

- Model: Kokoro-82M ONNX, loaded at startup, runs on CPU (frees GPU for VLM inference)
- Function: synthesize_speech(text: str) -> bytes — returns WAV buffer
- Endpoint: POST /api/synthesize — streamed back to the dispatcher audio widget
- Target: < 100 ms per sentence for real-time playback

4. Layer 3 — Agent Orchestration

The agent engine (backend/agents/civic_vox_graph.py) implements a LangGraph StateGraph. All agent nodes share an immutable snapshot of AgentState and write only to their designated output fields.

4.1 AgentState schema

TypedDict fields

messages: list[BaseMessage] · video_frames_base64: list[str] · gps_coordinates: dict · vision_analysis: dict · spatial_data_results: dict · next_step: str · final_dispatch_report: str

4.2 Node specifications

Node	Input fields consumed	Output fields written	LLM calls
Orchestrator	messages[-1] (transcript)	next_step, urgency_level	1 — text-only routing prompt
Vision Agent	video_frames_base64[-1], messages	vision_analysis, next_step	1 — multimodal (text + image_url)
Localizer Agent	gps_coordinates, vision_analysis	spatial_data_results, next_step	0 — pure GeoPandas spatial query
Report Compiler	messages, vision_analysis, spatial_data_results	final_dispatch_report, next_step	1 — synthesis prompt

4.3 Routing logic

Conditional edges are resolved by a `route_decision(state)` function reading `state.next_step`:

From	Condition	To
Orchestrator	<code>next_step == 'vision'</code>	Vision Agent
Orchestrator	<code>next_step == 'localizer' (no camera)</code>	Localizer Agent
Vision Agent	<code>always</code>	Localizer Agent
Localizer Agent	<code>always</code>	Report Compiler
Report Compiler	<code>always</code>	END

4.4 System prompts

Orchestrator prompt

```
Analyse the citizen report: "{transcript}"
Output ONLY valid JSON: { "requires_vision": bool, "urgency_level":
"LOW"|"HIGH"|"CRITICAL" }
```

Vision Agent prompt

```
Analyse this emergency video frame. Output ONLY valid JSON:
{ "hazard_type": str, "severity_scale": int 1-10, "water_depth_m": float|null,
  "structural_risk": bool, "location_cues": str }
```

Report Compiler prompt

```
You are Toronto Emergency Command. Given the following inputs, generate a
priority-coded dispatch protocol covering:
1. Threat classification and perimeter radius
2. Infrastructure access nodes (exact distances)
3. Building blueprint vulnerabilities
4. Recommended crew type and count
```

JSON safety note

Open-weight VLMs frequently wrap JSON output in ````json ... ```` markdown fences. All agent nodes must strip these before calling `json.loads()`. Use: `import re; clean = re.sub(r'```[\w]*\n?|```', '', raw).strip()`

5. Layer 4 — AI Inference & Data Infrastructure

5.1 Inference server

Option	Command	Notes
NVIDIA NIM (preferred)	<code>docker run --gpus all nvcr.io/nim/meta/llama-3.2-11b-vision-instruct</code>	Best GB10 tensor core utilisation
vLLM (fallback)	<code>python -m vllm.entrypoints.openai.api_server --model meta-llama/Llama-3.2-11B-Vision-Instruct</code>	Pip-installable, no Docker required
Ollama (dev/testing)	<code>ollama serve && ollama pull llama3.2-vision</code>	Easiest setup, lower throughput

All three options expose an OpenAI-compatible REST API at `http://localhost:8000/v1`. No agent code changes are required when switching between them — only the `LOCAL_LLM_URL` environment variable must be set.

5.2 Model inventory

Model	Task	Quant	VRAM	Latency target
Llama-3.2-11B-Vision-Instruct	Vision analysis + text generation	INT4 / FP8	~8 GB	< 60 ms/token
Whisper Large-v3	Speech-to-text	FP16	~3 GB	< 300 ms / 10 s
Kokoro-82M	Text-to-speech (ONNX)	FP32	< 1 GB (CPU)	< 100 ms / sentence

5.3 Toronto Open Data spatial module

All geospatial data is pre-loaded into memory once at server startup via `backend/data/toronto_loader.py`:

Dataset	Format	In-memory object	Query function
Fire Hydrant locations	GeoJSON	GeoDataFrame (EPSG:2958)	<code>get_closest_hydrants(lat, lng, n=3)</code>
RentSafeTO (apartments)	CSV → GeoDataFrame	GeoDataFrame (EPSG:2958)	<code>get_building_specs(lat, lng)</code>
Street Centrelines	GeoJSON	GeoDataFrame (EPSG:2958)	Nearest road segment lookup

Dataset	Format	In-memory object	Query function
311 Service Requests	CSV	Pandas DataFrame	Filter by address / date

- All layers are projected to NAD83 / UTM Zone 17N (EPSG:2958) for accurate metre-based distance calculations.
- An R-tree spatial index is built on the hydrants GeoDataFrame at startup for sub-millisecond nearest-neighbour lookup.
- Total estimated memory footprint of all datasets in-process: < 500 MB.

6. End-to-End Data Flow

The following table traces a single incident through every system component, from citizen camera stream to final dispatch report:

Step	Component	Action	Output
1	Mobile browser	Captures video frame, records audio, reads GPS	JPEG (base64), PCM blob, { lat, lng }
2	Frontend JS	Assembles payload, POSTs to /api/incident	HTTP request to FastAPI
3	FastAPI gateway	Validates payload via Pydantic; calls transcribe_audio(pcm)	Transcript string
4	LangGraph engine	Initialises AgentState; invokes civic_vox_engine.invoke(state)	Begins graph traversal
5	Orchestrator node	Sends transcript to VLM; parses routing JSON	next_step = 'vision'
6	Vision node	Sends frame_b64 + prompt to VLM via multimodal API call	vision_analysis dict
7	Localizer node	Calls get_closest_hydrants() + get_building_specs() via GeoPandas	spatial_data_results dict
8	Compiler node	Sends combined context to VLM; generates dispatch protocol text	final_dispatch_report string
9	FastAPI gateway	Serialises completed AgentState to JSON response	HTTP 200 + JSON payload
10	Frontend	Updates map markers, agent telemetry panel, report text window	Live dashboard update
11	Optional TTS	POST /api/synthesize with report text → Kokoro synthesizes WAV	Audio playback to dispatcher

7. Component Interface Contracts

These JSON schemas are the source of truth for cross-layer communication. All three engineers must agree on these before Hour 3 of the hackathon.

7.1 Frontend → Gateway

```
POST /api/incident
{
  "transcript": "There is flooding in the basement",
  "frame_b64": "<base64-encoded JPEG string>",
  "gps": { "lat": 43.6629, "lng": -79.3957 }
}
```

7.2 Gateway → Frontend (response)

```
{
  "report": "<dispatch protocol text>",
  "urgency": "CRITICAL",
  "vision": { "hazard_type": "Flooding", "severity_scale": 7, ... },
  "spatial": { "closest_hydrant": { "id": 1042, "distance_meters": 38.5, "status": "Operational" },
    "building_specs": { "floors": 12, "contact": "Toronto Housing Corp" } }
}
```

7.3 WebSocket telemetry event (per node completion)

```
{
  "node": "vision",
  "status": "complete",
  "timestamp": "2026-05-29T21:05:33.412Z"
}
```

8. Environment Variables

Variable	Default	Description
LOCAL_LLM_URL	http://localhost:8000/v1	OpenAI-compatible endpoint for the local inference server
LOCAL_LLM_MODEL	meta/llama-3.2-11b-vision-instruct	Model name passed in each chat/completions request

Variable	Default	Description
NEXT_PUBLIC_API_URL	http://192.168.x.x:8080	FastAPI URL — must be set to GB10 LAN IP for mobile demo
NEXT_PUBLIC_WS_URL	ws://192.168.x.x:8080/ws/stream	WebSocket URL for live telemetry
NEXT_PUBLIC_MAPBOX_TOKEN	pk.eyJ1lloi...	Free Mapbox public token for vector tile map
DATA_DIR	./backend/data	Path to Toronto Open Data GeoJSON / CSV files

9. Non-Functional Requirements

Requirement	Target	How achieved
End-to-end latency	< 100 ms (vision) / < 60 ms (text-only)	TensorRT-LLM INT4 on GB10; R-tree spatial index; in-memory data
Offline operation	100% during demo	All models and data pre-loaded; no external API calls
Vision throughput	≥ 10 FPS sustained	Single-pass INT4 VLM inference on GB10 tensor cores
STT accuracy (English)	< 8% WER	Whisper Large-v3 on-device
Spatial query speed	< 20 ms	GeoPandas R-tree index, EPSG:2958 projection pre-applied
Dashboard refresh	≤ 500 ms	WebSocket events pushed immediately after each agent node completes

10. GB10 Optimisation Notes

These optimisations should be demonstrated explicitly during the pitch to show hardware awareness:

- Pre-load all GeoDataFrames and the R-tree index into a global dict at FastAPI startup — not inside the request handler. Eliminates disk I/O from the critical path.
- Use TensorRT-LLM with FP8 or INT4 quantisation on the GB10 to saturate tensor cores. Show tokens-per-second counter in the dashboard.
- Pin the Whisper Large-v3 CUDA process to a dedicated CUDA stream to avoid competing with the VLM for memory bandwidth.
- Run Kokoro-82M TTS on CPU (ONNX runtime) — frees all 20 GB of GB10 VRAM for VLM inference while TTS completes in parallel.

- Use Python asyncio + FastAPI async endpoints for non-blocking I/O; the LangGraph `ainvoke()` call runs the agent graph as a coroutine, keeping the event loop free for WebSocket events.